

Introduction to Intelligent Robotics - Robot Control Homework

April 2026

Problem. Implementing Basic Control Algorithms

In this homework you will implement two classical control algorithms:

- a PID controller for the CartPole environment, and
- iterative Linear Quadratic Regulator (iLQR) for a 2-link planar arm.

We have provided a starter code package with the simulator APIs and the control loop entry points you need. Please implement your own controller logic in the indicated files. If you decide to deviate from the provided API, document your changes clearly in your submission.

You should not need a cluster or a GPU for this homework.

Part 1. PID Control for CartPole

In this part you will implement a PID controller to balance the CartPole system. Your controller should read the current CartPole observation and output the control action applied at the current step.

Implementation task:

- Complete the PID controller implementation in the provided starter code.
- Use the supplied runner script to connect your controller to the simulator loop.
- Tune your PID gains so that the pole stays balanced for as long as possible.

Suggested workflow:

- Start with proportional feedback on the pole angle.
- Then incorporate derivative feedback to reduce oscillation.
- Add integral action only if it helps with steady-state drift.
- Keep track of episode reward, survival time, and representative trajectories.

Report the following:

- The PID structure you used and the final gains.
- The total reward and number of steps before termination.
- Plots of cart position, cart velocity, pole angle, pole angular velocity, and control actions.
- A short discussion of how each PID term affected behavior.

Part 2. iLQR for a 2-Link Planar Arm

You will now implement iLQR for the planar dual-link arm simulator that we provide. The simulator follows the gym API with a few extra attributes and function arguments that make numerical dynamics approximation easier.

Environment

The environments define a cost matrix Q and a cost matrix R . Use these when computing your trajectories. The rewards returned by the environment are derived from the same quadratic cost.

The step function includes an additional argument `dt`. When estimating dynamics with finite differences, you will typically use a much smaller `dt` than the default simulation step. When you are executing your planned control sequence in the simulator, use the normal simulation step.

You can also explicitly set the simulator state using the `state` attribute. This is useful when approximating local dynamics around a nominal trajectory.

There are several environment variants summarized in Table 1.

Environment Name	Description
TwoLinkArm-v0	Two link planar arm with fixed goal. Simulation allows infinite torque values. The reward uses a high penalty for being away from the goal and a low penalty for controls.
TwoLinkArm-limited-torque-v0	Same as TwoLinkArm-v0 but the torques are limited.
TwoLinkArm-random-goal-v0	Same as TwoLinkArm-v0 but the goal changes on each reset.
TwoLinkArm-limited-torque-random-goal-v0	Same as TwoLinkArm-random-goal-v0 but the torque values are clipped.
TwoLinkArm-v1	Same as TwoLinkArm-v0 but with high cost for control input.
TwoLinkArm-limited-torque-v1	Same as TwoLinkArm-limited-torque-v0 but with high cost for control input.
TwoLinkArm-random-goal-v1	Same as TwoLinkArm-random-goal-v0 but with high cost for control input.
TwoLinkArm-limited-torque-random-goal-v1	Same as TwoLinkArm-limited-torque-random-goal-v0 but with high cost for control input.

Table 1: Descriptions of the different environments.

iLQR

Implement iLQR using the provided code templates and then perform the following experiments. iLQR will take longer to run than PID control, but it should still run on a standard laptop.

Preliminary settings:

- Set the control horizon to $tN = 100$.
- Use an intermediate cost of $\|u\|^2$ for steps $1 \sim tN - 1$.
- Use a final cost of $10^4 \times \|x_{tN} - x^*\|^2$.
- Set the maximum number of optimization iterations to 10^5 .
- You may add any reasonable early stopping criterion if the optimization converges.

Suggestions:

- If matrix inversions become numerically unstable, add λI with $\lambda > 0$.
- Start from simple initial states and fixed-goal environments for debugging.
- Make sure your forward rollout, linearization, and backward pass are each validated independently.

Experiments:

- Test your iLQR implementation on the TwoLinkArm-v0 environment. Record the total reward and number of steps to reach the goal. Also plot q , $\dot{q}$, and your control inputs u .
- Test your iLQR implementation on the TwoLinkArm-limited-torque-v0 environment. Record the total reward and number of steps to reach the goal. Also plot q , $\dot{q}$, and your control inputs u . Additionally, plot u clipped to the action space of this environment.
- Compare the performance of your controller on these two environments.
- Test your iLQR implementation on the TwoLinkArm-v1 environment. Record the total reward and number of steps to reach the goal. Also plot q , $\dot{q}$, and your control inputs u .
- Test your iLQR implementation on the TwoLinkArm-limited-torque-v1 environment. Record the total reward and number of steps to reach the goal. Also plot q , $\dot{q}$, and your control inputs u . Additionally, plot u clipped to the action space of this environment.
- Compare the performance on the v1 environments to the corresponding v0 environments.

(Optional) Speed Up iLQR

iLQR is often slower than we would like. Try to improve convergence in any way you can justify. Potential directions include changing the cost function, improving the line search, or using a better initialization strategy.

Submission

A starting codebase is provided. You may modify any component of this codebase, including but not limited to the algorithmic implementation and the environment configuration. Modifications to the environment must preserve reasonable and valid system dynamics.

Version control is managed using Git. Your submission should therefore include a patch file capturing all changes relative to the original codebase. This patch can be generated using `git diff > mypatch.patch`.

Enjoy coding!